

K-means Clustering

gaurav sood
http://gsood.com
twitter | github

Unsupervised Learning

Unsupervised Learning

- Unsupervised learning finds structure without outcome labels

Unsupervised Learning

- Unsupervised learning finds structure without outcome labels
- In supervised learning, labels guide prediction

Unsupervised Learning

- Unsupervised learning finds structure without outcome labels
- In supervised learning, labels guide prediction
- Regression estimates a relationship between predictors and an outcome

Unsupervised Learning

- Unsupervised learning finds structure without outcome labels
- In supervised learning, labels guide prediction
- Regression estimates a relationship between predictors and an outcome
- Supervised Learning $\sim$ Given Apples and Oranges, learn traits of Apples Vs. Oranges

Unsupervised Learning

- Unsupervised learning finds structure without outcome labels
- In supervised learning, labels guide prediction
- Regression estimates a relationship between predictors and an outcome
- Supervised Learning $\sim$ Given Apples and Oranges, learn traits of Apples Vs. Oranges
- Given a bunch of spherical fruits, optimally describe types of fruits

Ways to Think About Unsupervised Learning

Ways to Think About Unsupervised Learning

- Learning the probability model of the data $p(x_n | x_1, \ldots, x_{n-1})$

Ways to Think About Unsupervised Learning

- Learning the probability model of the data $p(x_n|x_1, \ldots, x_{n-1})$
- **Applications:** Outlier detection, Data compression

Ways to Think About Unsupervised Learning

- Learning the probability model of the data $p(x_n|x_1, \ldots, x_{n-1})$
- **Applications:** Outlier detection, Data compression
- Find rows similar to each other, groups of rows dissimilar to each other

Ways to Think About Unsupervised Learning

- Learning the probability model of the data $p(x_n|x_1, \ldots, x_{n-1})$
- **Applications:** Outlier detection, Data compression
- Find rows similar to each other, groups of rows dissimilar to each other
- Find columns similar to each other, groups of columns dissimilar to each other

Ways to Think About Unsupervised Learning

- Learning the probability model of the data $p(x_n|x_1, \ldots, x_{n-1})$
- **Applications:** Outlier detection, Data compression
- Find rows similar to each other, groups of rows dissimilar to each other
- Find columns similar to each other, groups of columns dissimilar to each other
- **Applications:** Group movies by ratings, Segment shoppers

Solutions

- Two kinds of methods:

Solutions

- Two kinds of methods:
    - Principal components analysis

Solutions

- Two kinds of methods:
  - Principal components analysis
  - Clustering

Solutions

- Two kinds of methods:
  - Principal components analysis
  - Clustering
- Clustering looks to partition data into similar subgroups

Solutions

- Two kinds of methods:
  - Principal components analysis
  - Clustering
- Clustering looks to partition data into similar subgroups
- Two popular methods:

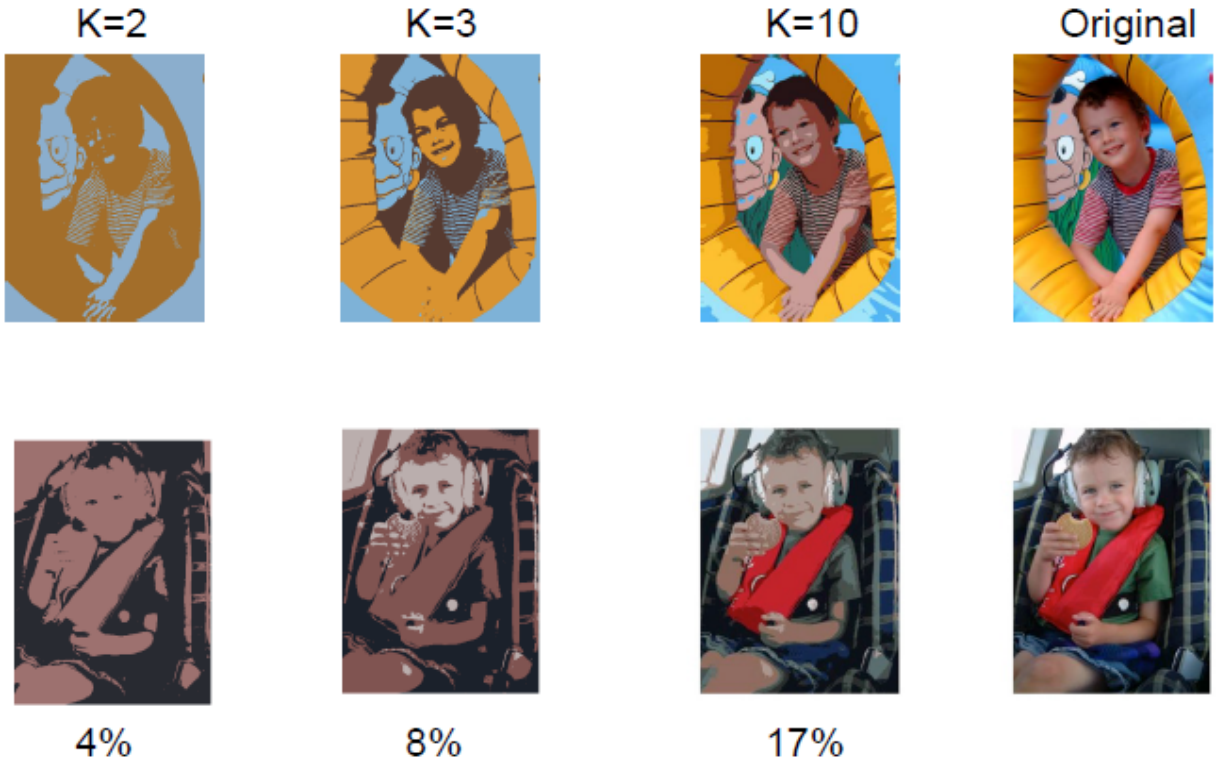
Solutions

- Two kinds of methods:
  - Principal components analysis
  - Clustering
- Clustering looks to partition data into similar subgroups
- Two popular methods:
  - Hierarchical clustering (computationally expensive)

Solutions

- Two kinds of methods:
  - Principal components analysis
  - Clustering
- Clustering looks to partition data into similar subgroups
- Two popular methods:
  - Hierarchical clustering (computationally expensive)
  - *k*-means clustering (pre-specify *k*)

Source: Pattern Recognition and Machine Learning



k-Means Clustering

- *k*-means: Assume that we must split data into *k* clusters

k-Means Clustering

- *k*-means: Assume that we must split data into *k* clusters
  - Each observation belongs to one cluster

k-Means Clustering

- *k*-means: Assume that we must split data into *k* clusters
  - Each observation belongs to one cluster
  - No observation belongs to more than one cluster

k-Means Clustering

- *k*-means: Assume that we must split data into *k* clusters
  - Each observation belongs to one cluster
  - No observation belongs to more than one cluster
- Find partitioning that minimizes within cluster variation summed over all *k* clusters

k-Means Clustering

- k-means: Assume that we must split data into k clusters
  - Each observation belongs to one cluster
  - No observation belongs to more than one cluster
- Find partitioning that minimizes within cluster variation summed over all k clusters
- Minimize squared Euclidean distance to cluster means. Equivalently:

$$\min_{C_1,\ldots,C_K} \sum_{k=1}^{K} \frac{1}{2|C_k|} \sum_{i,i' \in C_k} \sum_{j=1}^{p} (x_{ij} - x_{i'j})^2 \qquad (1)$$

k-Means (Lloyd's) Algorithm

k-Means (Lloyd's) Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty

k-Means (Lloyd's) Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:

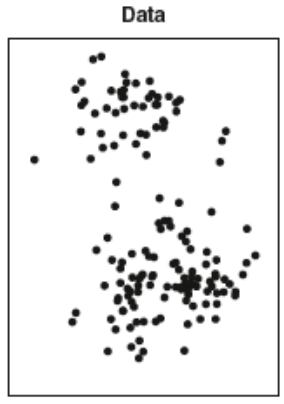
k-Means (Lloyd's) Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid

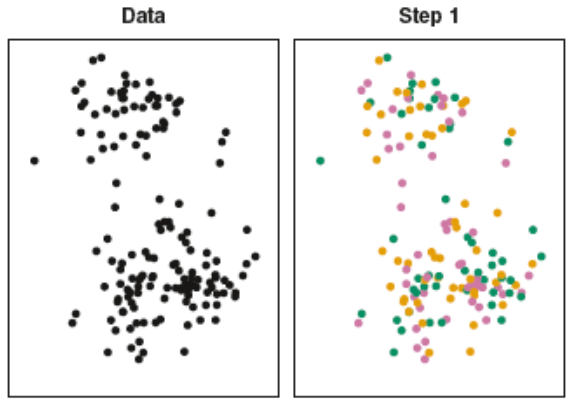
k-Means (Lloyd's) Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest

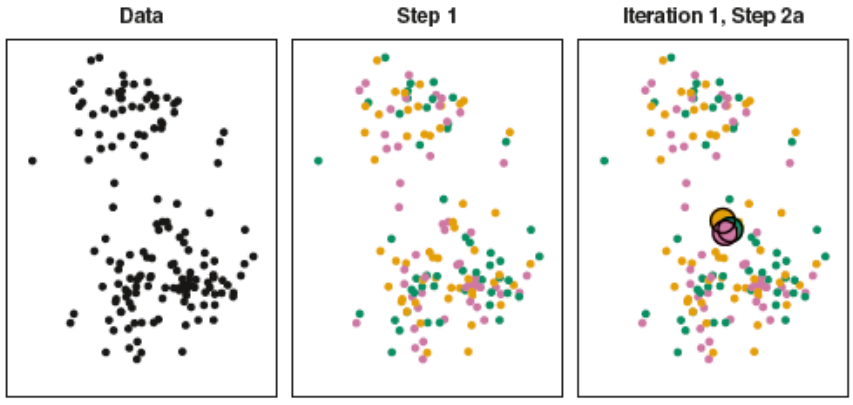
Source: James et al. 2015



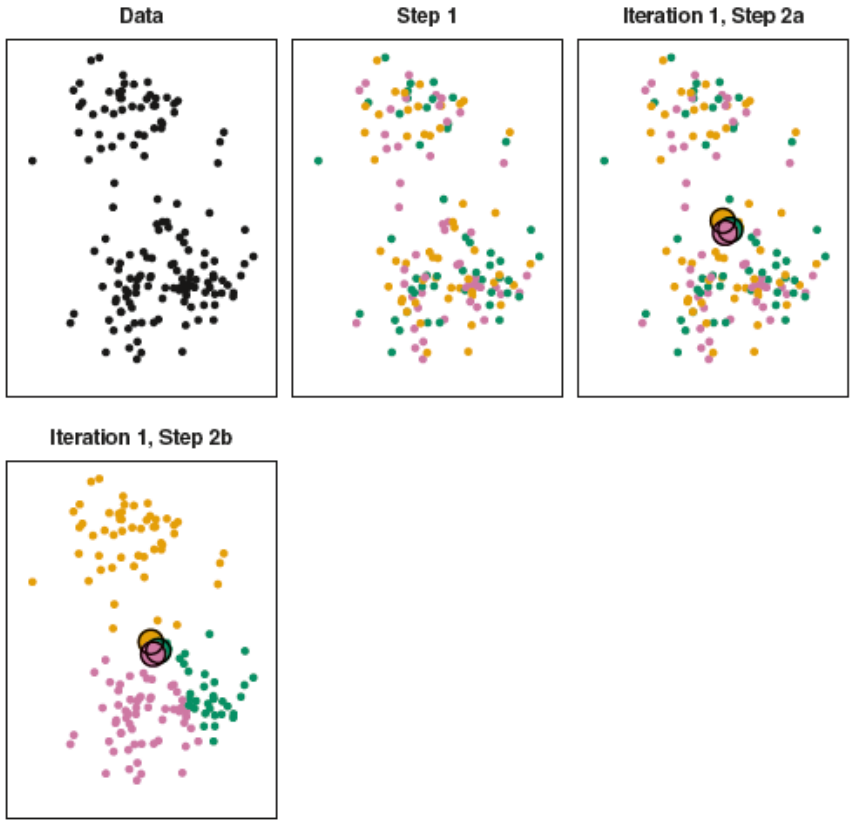
Source: James et al. 2015



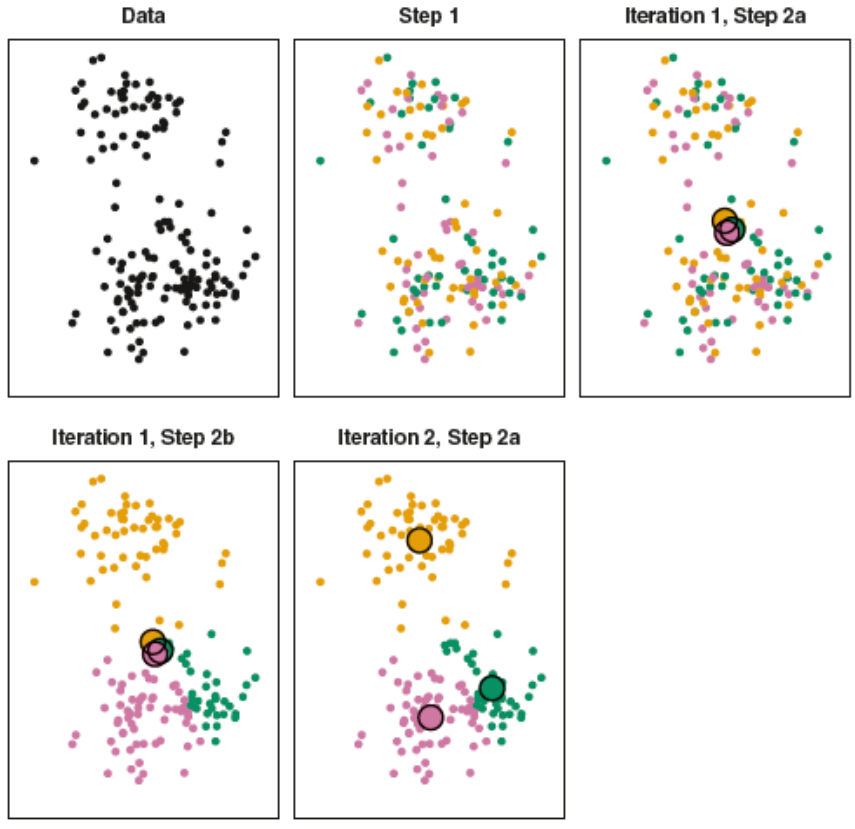
Source: James et al. 2015



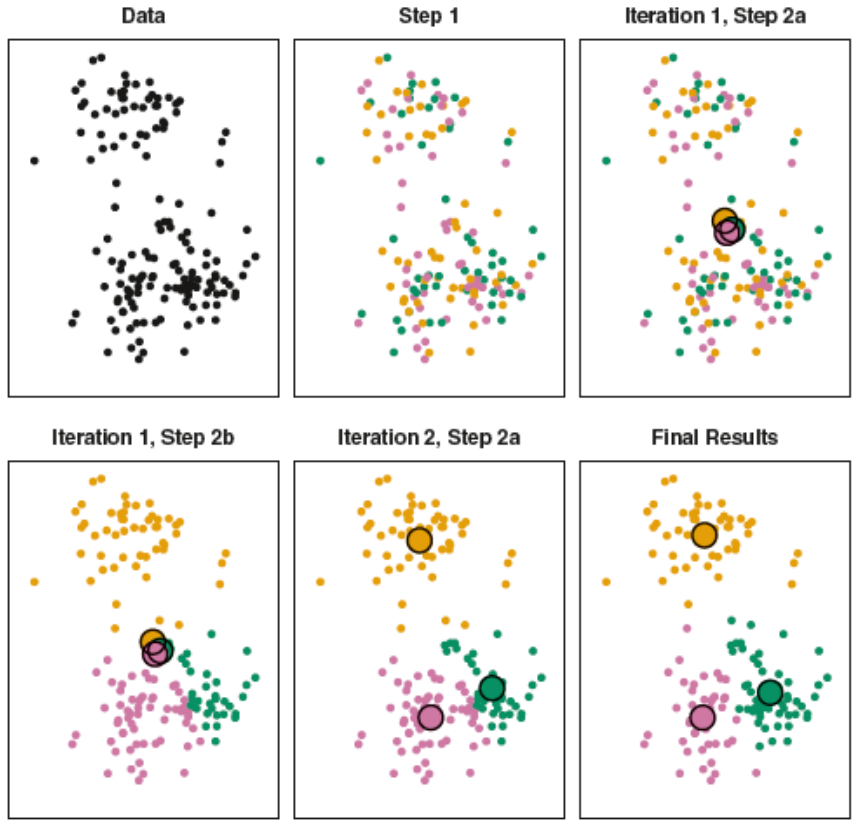
Source: James et al. 2015



Source: James et al. 2015



Source: James et al. 2015



k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest

k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest
- Why does it work?

k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest
- Why does it work?
- Each step cannot increase the objective, but a global minimum is not guaranteed.

k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest
- Why does it work?
- Each step cannot increase the objective, but a global minimum is not guaranteed.
- Initialization:

k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest
- Why does it work?
- Each step cannot increase the objective, but a global minimum is not guaranteed.
- Initialization:
  - Forgy: Randomly choose *k* observations and set them as centroids.

k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest
- Why does it work?
- Each step cannot increase the objective, but a global minimum is not guaranteed.
- Initialization:
  - Forgy: Randomly choose *k* observations and set them as centroids.
  - Random partition: initialize nonempty clusters. Use several starts.

k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest
- Why does it work?
- Each step cannot increase the objective, but a global minimum is not guaranteed.
- Initialization:
  - Forgy: Randomly choose *k* observations and set them as centroids.
  - Random partition: initialize nonempty clusters. Use several starts.
  - For an empty cluster, reinitialize its centroid to a data point.

k-Means Algorithm

- Initialize *k* distinct centroids; ensure clusters remain nonempty
- Iterate:
  - For each of the *k* clusters, compute the centroid
  - Assign each observation to cluster whose centroid is closest
- Why does it work?
- Each step cannot increase the objective, but a global minimum is not guaranteed.
- Initialization:
  - Forgy: Randomly choose *k* observations and set them as centroids.
  - Random partition: initialize nonempty clusters. Use several starts.
  - For an empty cluster, reinitialize its centroid to a data point.
  - Pick dispersed points as centroids. For e.g. *k*-means++ and variations of it.

Distance between clusters

- Complete Linkage
  Farthest distance between points in clusters

Distance between clusters

- Complete Linkage
  Farthest distance between points in clusters
- Single
  Closest pair

Distance between clusters

- Complete Linkage
  Farthest distance between points in clusters
- Single
  Closest pair
- Average
  All pairs, and then take the average

Distance between clusters

- Complete Linkage
  Farthest distance between points in clusters
- Single
  Closest pair
- Average
  All pairs, and then take the average
- Centroid
  Has problems called inversions
  Used in Genomics

Distance between clusters

- Complete Linkage
  Farthest distance between points in clusters
- Single
  Closest pair
- Average
  All pairs, and then take the average
- Centroid
  Has problems called inversions
  Used in Genomics
- Complete and Average most commonly used

Practical Issues

- Choice of Similarity Measure

Practical Issues

- Choice of Similarity Measure
    - Scaling Matters

Practical Issues

- Choice of Similarity Measure
  - Scaling Matters
  - Jaccard similarity can be estimated using minhash

Practical Issues

- Choice of Similarity Measure
  - Scaling Matters
  - Jaccard similarity can be estimated using minhash
  - Other distances require appropriate methods, e.g. *k*-medoids, not the ordinary *k*-means mean update

Practical Issues

- Choice of Similarity Measure
  - Scaling Matters
  - Jaccard similarity can be estimated using minhash
  - Other distances require appropriate methods, e.g. *k*-medoids, not the ordinary *k*-means mean update
- For high-dimensional data, consider scaling and PCA; DANN requires labels.

Practical Issues

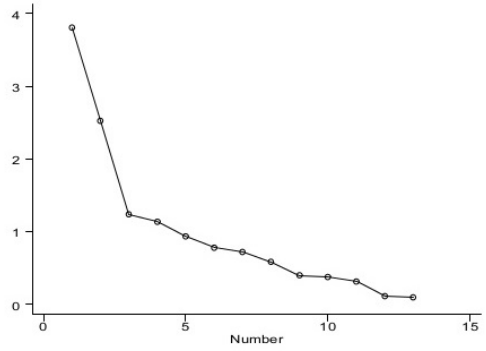
- Choice of Similarity Measure
  - Scaling Matters
  - Jaccard similarity can be estimated using minhash
  - Other distances require appropriate methods, e.g. *k*-medoids, not the ordinary *k*-means mean update
- For high-dimensional data, consider scaling and PCA; DANN requires labels.
- Choosing *k*:

Practical Issues

- Choice of Similarity Measure
  - Scaling Matters
  - Jaccard similarity can be estimated using minhash
  - Other distances require appropriate methods, e.g. *k*-medoids, not the ordinary *k*-means mean update
- For high-dimensional data, consider scaling and PCA; DANN requires labels.
- Choosing *k*:
  - Calculate total within-cluster squared distances for multiple *k*

Practical Issues

- Choice of Similarity Measure
  - Scaling Matters
  - Jaccard similarity can be estimated using minhash
  - Other distances require appropriate methods, e.g. *k*-medoids, not the ordinary *k*-means mean update
- For high-dimensional data, consider scaling and PCA; DANN requires labels.
- Choosing *k*:
  - Calculate total within-cluster squared distances for multiple *k*
  - Plot them, look for the *knee*



A(N)alyst choose k contd.

- With cluster sizes n_k, means $\bar{x}_k$, and overall mean $\bar{x}$:

$$B_K = \sum_{k=1}^{K} n_k \|\bar{x}_k - \bar{x}\|^2, \quad W_K = \sum_{k=1}^{K} \sum_{i \in C_k} \|x_i - \bar{x}_k\|^2.$$

- Calinski–Harabasz index:

$$\mathrm{CH}(K) = \frac{B_K/(K-1)}{W_K/(n-K)}, \quad 1 < K < n,\ W_K > 0.$$

- Larger values favor greater separation relative to within-cluster spread.

Choosing K: Gap Statistic

- Compare W_K to dispersion W_{Kb}^* from B simulated reference datasets:

$$\mathrm{Gap}(K) = \frac{1}{B}\sum_{b=1}^{B} \log W_{Kb}^* - \log W_K.$$

- Specify the reference distribution, often uniform over a bounding box in rotated coordinates.
- Let $s_K = \mathrm{sd}_b(\log W_{Kb}^*)\sqrt{1 + 1/B}$.
- One-standard-error rule: choose the smallest K such that

$$\mathrm{Gap}(K) \geq \mathrm{Gap}(K+1) - s_{K+1}.$$

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.
- With consistent tie handling, Lloyd iterations terminate; worst cases can take exponentially many steps.

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.
- With consistent tie handling, Lloyd iterations terminate; worst cases can take exponentially many steps.
- In practice, use a convergence tolerance and an iteration limit.

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.
- With consistent tie handling, Lloyd iterations terminate; worst cases can take exponentially many steps.
- In practice, use a convergence tolerance and an iteration limit.
- Big (Long) Data Solutions:

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.
- With consistent tie handling, Lloyd iterations terminate; worst cases can take exponentially many steps.
- In practice, use a convergence tolerance and an iteration limit.
- Big (Long) Data Solutions:
  - Bradley-Fayyad-Reina (BFR)

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.
- With consistent tie handling, Lloyd iterations terminate; worst cases can take exponentially many steps.
- In practice, use a convergence tolerance and an iteration limit.
- Big (Long) Data Solutions:
  - Bradley-Fayyad-Reina (BFR)
  - CURE

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.
- With consistent tie handling, Lloyd iterations terminate; worst cases can take exponentially many steps.
- In practice, use a convergence tolerance and an iteration limit.
- Big (Long) Data Solutions:
  - Bradley-Fayyad-Reina (BFR)
  - CURE
- BFR
  - Summarizes roughly Gaussian clusters with axis-aligned covariance.

Running Time

- $O(nKp)$ per Lloyd iteration for n points, K clusters, p features.
- With consistent tie handling, Lloyd iterations terminate; worst cases can take exponentially many steps.
- In practice, use a convergence tolerance and an iteration limit.
- Big (Long) Data Solutions:
  - Bradley-Fayyad-Reina (BFR)
  - CURE
- BFR
  - Summarizes roughly Gaussian clusters with axis-aligned covariance.
  - Uses cluster summaries to assess standardized distance to a centroid